No.	Django Questions
	What is Django?
	Django:- Latest version
	Why is Django Used/Key Features?
	Django Architecture (MVT)
	What is MTV architecture?
	Create a Django Project?
	Django Project Structure
	what is manage.py?
	What is Django Apps?
	Create app
	Project vs App

No.	Django Models & Database Questions
	What is a Model?

🎯 Dajngo project setup

1. Virtual Environment

```
# Create virtual env
python -m venv venv

# activate
venv\Scripts\activate
```

2. Django install karo

```
pip install django

# check version
django-admin --version
```

3. Django project create karo

```
django-admin startproject myproject

myproject/
│
├── manage.py
│
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

4. Server run karo

```
python manage.py runserver  # http://127.0.0.1:8000/

# Django ka default page open ho jayega.
```

5. App create karo

```
python manage.py startapp users

myproject/
│
├── manage.py
│
├── myproject/
│   ├── settings.py
│   ├── urls.py
│   └── ...
│
└── users/
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    └── views.py
```

6. App ko settings.py mein add karo

- myproject/settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'users',    # add app
]
```

7. First View banao

- users/views.py

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Hello Django")
```

8. URL seeting

- App ke andar urls.py banao:- users/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

- Project ke urls.py mein include karo

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('users.urls')),
]
```

9. Database migrations

```
python manage.py migrate
```

10. Admin user banao

```
python manage.py createsuperuser

# http://127.0.0.1:8000/admin/
```

🎯 Ques. What is Django?

- Django—pronounced **“Jango”**.
- Django is a free and open source and server-side web application framework written in Python.
- Django follows the **MVT** (Model View Template) pattern which is based on the Model View Template architecture. and provides many built-in features like authentication, database management, security, and an admin panel.
- It was orginally created By **Adrian Holovaty** and **simon willison**.

latest version of Django?

- The latest version of Django is Django 4.1.

Why is Django Used/Key Features?

- Rapid development
- Built-in Admin Panel
- Authentication & Authorization
- ORM (Object Relational Mapper)
- Security features (CSRF, XSS, SQL Injection protection)
- Scalable and reusable code
- Large community support
- Form Handling
- Session Management
- Middleware Support
- REST API support (using Django REST Framework)

🎯 Django Architecture (MVT)

- url request -> manage.py -> setting.py ->urls.py -> views.py -> models.py -> template.
- Django follows a software design pattern called a **MVT(Model view Template)** architecture.
- **Model:-** It helps in **handling the databse**. they provide the option to create edit and query data records in the databse.
- **View:-** the view is used to **execute the business logic** and intrect with a model to carry data and renders a template.
- **Template:-** The template is a **presentation layer**. It define the structure of file layout to present data in web page. it is an **html file** mixed with django template language.

```
User Request
     ↓
   View
     ↓
  Model ⇔ Database
     ↓
 Template
     ↓
User Response
```

🎯 What is MTV architecture?

- Model → Handles database tables and data
- Template → Handles UI (HTML)
- View → Contains business logic

🎯 Create a Django Project?

```
django-admin startproject myproject

myproject/
│
├── manage.py
│
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

🎯 Django Project Structure?

- When we create a Django project using:

```
# Django creates a structure like this:

myproject/
│
├── manage.py
│
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

1. manage.py

- This is the **command-line utility** used to manage your Django project.

```
# Examples:
python manage.py runserver  :-  Start development server
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py startapp users :-   Create an app
python manage.py shell
```

2. settings.py

- Contains the **configuration/settings** of your Django project.
- It contains things like:
    - Database configuration
    - Installed apps
    - Middleware
    - Templates
    - Static files
    - Media files
    - Security settings
    - Time zone

```
INSTALLED_APPS = [
    'users',
    'products',
]
```

```
DATABASES = {
    # Database configuration
}
```

3. urls.py

- This is the main URL configuration of the project.
- It decides which view should handle a particular URL.

```
# Example:

from django.urls import path
from users import views

urlpatterns = [
    path('users/', views.users),
]
```

4. models.py

- Technically, the default project package created by startproject does not contain models.py; models.py belongs to Django apps.

```
# For example:

myproject/
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
│
└── users/
    ├── models.py
    ├── views.py
    └── ...
```

- models.py defines your database models.

```
class User(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
```

5. views.py

- Also belongs to an app, not normally the project package.

- It contains the application/business logic that handles requests.

```
def home(request):
    return HttpResponse("Hello World")
```

6. asgi.py

- ASGI stands for Asynchronous Server Gateway Interface.
- It is used to serve Django applications in asynchronous environments and with ASGI-compatible servers.

7. wsgi.py

- WSGI stands for Web Server Gateway Interface.
- It is commonly used to deploy Django applications with traditional synchronous WSGI servers.

Project + App Structure

- In a real project, you'll usually have:

```
myproject/
│
├── manage.py
│
├── myproject/              # Project configuration
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── users/                  # Django App
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
│
└── products/               # Another Django App
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    └── views.py
```

🎯 Django Apps?

- A Django app is a small, independent module of a Django project that handles one specific functionality.
- A Django app is a reusable and modular component of a Django project that is responsible for a specific functionality, such as users, products, orders, or payments. A Django project can contain multiple apps.

```
E-commerce Project
│
├── users        → Login, registration, profiles
├── products     → Product management
├── orders       → Order management
├── payments     → Payment processing
└── reviews      → Product reviews
```

🎯 Creating an App

```
python manage.py startapp products


products/
├── migrations/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
├── views.py
```

🎯 Project vs App

Django Project	Django App
Complete website/application	One specific functionality
Contains multiple apps	Handles a particular feature
Created using `django-admin startproject`	Created using `python manage.py startapp`
Example: `ecommerce`	Example: `products`, `orders`

🎯 What is a Model?

- A Django Model is a Python class that represents a database table.
- It defines the fields and relationships of the data and allows us to interact with the database using Django ORM without writing raw SQL for most operations.

Where do we create a Model?

- Models are generally created inside an app's models.py:

```
myproject/
│
├── manage.py
│
├── myproject/
│   ├── settings.py
│   └── urls.py
│
└── users/
    ├── models.py      # ← Model is created here
    ├── views.py
    └── admin.py
```

```
from django.db import models

class User(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    age = models.IntegerField()
```

- After creating the model, run:

```
python manage.py makemigrations

# Then
python manage.py migrate
```

What are Migrations?

- Migrations are files that Django uses to track and apply changes made to models in the database schema.
- Whenever you create, modify, or delete a model field, Django records those changes in migration files.

🎯 Difference between makemigrations and migrate?

- makemigrations creates migration files based on changes in Django models, while migrate applies those migration files to the database and updates the database schema.
- makemigrations prepares the changes; migrate executes them.

makemigrations	migrate
Creates migration files	Applies migration files to the database
Detects changes in models	Executes SQL on the database
Does not change the database	Changes the database structure
Generates migration scripts	Runs migration scripts

```
# Create/Modify a Model
from django.db import models

class Employee(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()

# Step 2
python manage.py makemigrations # Run makemigrations

# Example migration file:
migrations.CreateModel(
    name='Employee',
    fields=[
        ('id', models.BigAutoField(primary_key=True)),
        ('name', models.CharField(max_length=100)),
        ('email', models.EmailField()),
    ],
)

# Run migrate
python manage.py migrate
```

🎯 What is the Meta Class?

- The Meta class is an inner class inside a Django model that is used to provide metadata (extra configuration) about the model.
- It does not create database fields. Instead, it controls how Django behaves with the model.
- **HINDI:-** Meta class model ki additional configuration define karne ke liye use hoti hai. Isme db_table, ordering, verbose_name, unique_together, indexes, constraints, permissions jaise options define kiye ja sakte hain.

```
from django.db import models

class Employee(models.Model):
    name = models.CharField(max_length=100)
```

```
    email = models.EmailField()

    class Meta:
        db_table = "employees"
```

Common Uses of Meta

1. Specify a **custom table name**.

```
class Meta:
    db_table = "employees"

# appname_employee
```

2. ordering

- - (minus sign) lagane se descending order ho jata hai.
- for Ascending Order (A → Z)

```
class Meta:
    ordering = ['name']
```

- for Descending Order (Z → A)

```
class Meta:
    ordering = ['-name']
```

- ISI trahe se bahute sare hai

```
from django.db import models
from django.db.models import Q

class Employee(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    salary = models.DecimalField(max_digits=10, decimal_places=2)

    class Meta:
        db_table = "employees"
        ordering = ["name"]
        verbose_name = "Employee"
        verbose_name_plural = "Employees"
        unique_together = ['name', 'email']
        get_latest_by = "id"
        indexes = [
```

```
            models.Index(fields=["email"]),
        ]
        constraints = [
            models.CheckConstraint(
                condition=Q(salary__gte=0),
                name="salary_positive"
            )
        ]
```

Option	Purpose
`db_table`	Custom table name
`ordering`	Default sorting
`verbose_name`	Singular name in Admin
`verbose_name_plural`	Plural name in Admin
`unique_together`	Multiple fields unique
`indexes`	Database indexes
`constraints`	Custom DB constraints
`permissions`	Custom permissions
`default_permissions`	Add/remove default permissions
`managed`	Whether Django manages the table
`abstract`	Create abstract base model
`proxy`	Create proxy model
`app_label`	Assign model to an app manually
`get_latest_by`	Latest object field
`db_table_comment`	Add table comment (newer Django versions)

🎯 What is ORM?

- ORM (Object Relational Mapping) is a technique that allows us to interact with a database using Python objects and methods instead of writing SQL queries directly.
- Django provides a built-in ORM called Django ORM.

Advantages of ORM

- No need to write SQL for common operations.
- Database-independent (MySQL, PostgreSQL, SQLite, etc.).
- Faster development.
- More readable and maintainable code.
- Helps prevent SQL injection in normal ORM usage.

Example

1. Get Data

```
employees = Employee.objects.all()
Employee.objects.get(id=1)
Employee.objects.filter(name="Mohit")
```

2. Create

```
Employee.objects.create(
    name="Mohit",
    email="mohit@gmail.com"
)
```

3. Read

```
Employee.objects.all()
Employee.objects.get(id=1)
Employee.objects.filter(name="Mohit")
```

4. Update

```
emp = Employee.objects.get(id=1)
emp.name = "Rahul"
emp.save()
```

5. Delete

```
emp = Employee.objects.get(id=1)
emp.delete()
```

🎯 What is indexes in Django?

- Django mein database index add karne ke liye mainly Meta class ke andar indexes option use karte hain.
- Index ka purpose database queries ko faster banana hota hai, especially jab kisi column par frequently filter(), order_by() ya lookup kiya jata hai.

1. Single-field Index

```
from django.db import models

class Employee(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()

    class Meta:
        indexes = [
            models.Index(fields=['email']),
        ]
# Yahan email column par index create hoga.
```

- Single field ke liye directly:

```
email = models.EmailField(db_index=True)
```

2. Multiple Indexes

```
class Employee(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    department = models.CharField(max_length=100)

    class Meta:
        indexes = [
            models.Index(fields=['email']),
            models.Index(fields=['department']),
        ]
```

3. Composite Index

```
class Employee(models.Model):
    name = models.CharField(max_length=100)
    department = models.CharField(max_length=100)
    salary = models.IntegerField()

    class Meta:
        indexes = [
            models.Index(fields=['department', 'salary']),
        ]
```

4. Index ko Custom Name dena

```
class Meta:
    indexes = [
        models.Index(
            fields=['email'],
            name='employee_email_idx'
        ),
    ]
```

db_index=True vs Meta.indexes

db_index=True	Meta.indexes
Simple single-field index	More flexible
Field ke andar define hota hai	Meta ke andar define hota hai
email = models.EmailField(db_index=True)	models.Index(fields=['email'])
Basic use case	Composite/custom indexes ke liye useful

How do you write raw SQL?

- Django normally ORM use karta hai, lekin jab complex query ho ya ORM se query likhna convenient na ho, tab raw SQL use kar sakte hain.
- Django mein raw SQL execute karne ke mainly 3 common ways hain.

1. **Model.objects.raw()**

```
employees = Employee.objects.raw(
    "SELECT * FROM employees WHERE salary > %s",
    [50000]
)

for employee in employees:
    print(employee.name)
```

2. **connection.cursor()**

- Agar INSERT, UPDATE, DELETE ya koi arbitrary SQL execute karna hai:

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        "UPDATE employees SET salary = %s WHERE id = %s",
        [60000, 1]
    )
```

```
# Example select
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        "SELECT id, name FROM employees WHERE salary > %s",
        [50000]
    )

    rows = cursor.fetchall()

for row in rows:
    print(row)
```

3. RawSQL

- Django ke queryset ke andar custom SQL expression use karna ho to RawSQL use kar sakte hain:

```
from django.db.models.expressions import RawSQL

employees = Employee.objects.annotate(
    custom_value=RawSQL(
        "salary * 2",
        []
    )
)
```